

Reviewer Pack — Metric Dispersion Lower Bound for Monotone DNF Depth

Entry 735d4d6f849f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 17:48:53 UTC

Metric Dispersion Lower Bound for Monotone DNF Depth

Entry ID: 735d4d6f849f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 17:48:53 UTC

1. Conjecture

Field A (mathematical branch): Metric Geometry **Field B** (complexity object): Monotone DNF Formulas

Statement:

For any monotone DNF formula F with m terms, the dispersion $\mu(F) = \max_{i \neq j} \|x_i - x_j\|_2^2$ over term vectors $x_i \in \{0,1\}^n$ satisfies $\mu(F) \leq O(\log m)$ for $m \leq \text{poly}(n)$, but $\mu(k\text{-CLIQUE}) \geq \Omega(n^2)$ for all $k \geq 3$.

Rationale (proposer's reasoning):

High dispersion in term vectors forces geometric separation, making it infeasible to represent $k\text{-CLIQUE}$ with sparse DNF. Metric geometry provides a quantitative lens to bound term interactions via packing arguments.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3d3d664b6a912cb6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=1.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(m: int, n: int):
        dnf = []
        for _ in range(m):
            term = [random.randint(0, 1) for _ in range(n)]
            dnf.append(term)
        return dnf

    def euclidean_distance(x, y):
        return sum((xi - yi) ** 2 for xi, yi in zip(x, y))

    def dispersion(dnf):
        n = len(dnf[0])
        max_dist_squared = 0
        for i in range(len(dnf)):
            for j in range(i + 1, len(dnf)):
                dist_squared = euclidean_distance(dnf[i], dnf[j])
                if dist_squared > max_dist_squared:
                    max_dist_squared = dist_squared
        return max_dist_squared

    n_max = 40
    m_max = 40
    instances_tested = 30
    metric_name = "dispersion"

    results = []
    for _ in range(instances_tested):
        m = random.randint(5, min(m_max, n_max))
        dnf = generate_dnf(m, n_max)
        mu = dispersion(dnf)
        results.append(mu)

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))

    conjecture_holds = all(mu <= 5 * math.log(m) for m, mu in zip([m] * instances_tested, results))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
```

```

        "metric_name": metric_name,
        "metric_value": mean,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'counterexample': 'mapping_undefined'
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.733333333333334, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.566666666666666, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.133333333333333, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.0, 'instances_tested': 30, 'conjecture_holds
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.3, 'instances_tested': 30, 'conjecture_holds
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.766666666666666, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.333333333333332, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.933333333333334, 'instances_tested': 30, 'co
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.3, 'instances_tested': 30, 'conjecture_holds
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.4, 'instances_tested': 30, 'conjecture_holds
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.3, 'instances_tested': 30, 'conjecture_holds
TRIAL: {'metric_name': 'dispersion', 'metric_value': 28.266666666666666, 'instances_tested': 30, 'co
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test found counterexamples with dispersion values consistently exceeding $O(\log m)$ for $m \leq \text{poly}(n)$, directly contradicting the conjecture's upper bound | next: Investigate the 'mapping_undefined' counterexample to determine its concrete structure and validate its impact on the dispersion lower bound.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	76460
2	propose	ollama_remote	qwen3:8b	0	0	71237
3	preregistration	ollama_remote	qwen3:8b	0	0	24118
4	novelty	ollama_remote	qwen3:8b	0	0	20724
5	novelty	ollama_remote	qwen3:8b	0	0	18112
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16194
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7905
8	critic	ollama_remote	qwen3:8b	0	0	29971
9	judge	ollama_remote	qwen3:8b	0	0	15931

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 280654 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/735d4d6f849f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/735d4d6f849f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/735d4d6f849f.tar.gz` (if generated)